
TaxTrace

Configurable tax determination, validation and reconciliation engine.

Takes messy multi-ERP finance data, applies tax rules held as configuration rather than code, calculates expected tax, reconciles it against what the source system recorded, classifies every variance, and leaves an audit trail from any filed figure back to the source row and the rule version that produced it.

Built against a synthetic but deliberately realistic client, Meridian Commerce Group, operating across Australia, New Zealand, the United Kingdom and eight United States states.

Why the trail matters

Most tax data tooling can tell you *what* the tax should be. The harder question, and the one an auditor actually asks, is *why*. Every calculated row in TaxTrace carries the rule id, the rule version, the rate, and the timestamp that produced it. Given a filed number you can name the rule that made it.

Status

Stage	State
Business requirements	Complete
Synthetic data generator with known ground truth	Complete, tested
Data profiling	Day 1, in progress
Canonical model and SQL layer	Day 2
Rules engine	Day 3
Data quality and reconciliation	Day 4
Regression suite and incident response	Day 5
CI/CD, documentation, AI evaluation harness	Day 6

Quick start

```
git clone <repo>
cd taxtrace
python -m venv .venv && .venv\Scripts\activate      # Windows
pip install -r requirements.txt

python scripts/generate_data.py                    # writes data/, about 18 MB
python -m pytest -q                                # 20 tests
```

The dataset is **not committed**. It is regenerated deterministically from a seed, which is why data/ is in .gitignore. Same seed, same 50,300 rows, same 12,468 injected faults, every time.

The dataset

Four ERP extracts, each using its own field names, boolean conventions and rate formats, exactly as a real multi-system client would send them.

File	Rows	Field naming
erp_sap_au_transaction18,182	18,182	SAP (BELNR, MWSKZ, WMWST)
erp_dynamics_uk_transaction14,061.csv	14,061	Dynamics (DocumentNo, VATAmount)
erp_oracle_us_transaction12,876.csv	12,876	Oracle (TRX_NUMBER, TAX_AMT)
erp_netsuite_nz_transaction5,281.csv	5,281	NetSuite (tranid, taxtotal)

Plus customers, vendors, jurisdictions, tax codes, an effective-dated rate history, and a 250-row golden dataset.

Injected faults

12,468 faults across 20 fault types, logged by transaction id in an answer key so a pipeline's precision and recall can actually be measured.

Tier	Count	Findable by
LOUD	6,025	Profiling
MEDIUM	2,792	Referential integrity and business rules
SUBTLE	3,651	Reconciliation and statistics only

The SUBTLE tier is the interesting one: tax understated by a fraction of a per cent, banker's rounding where half-up was required, an exemption flag set without a valid certificate, a gross amount reported as net.

Architecture

```
4 ERP extracts → ingest → profile → stage → map to canonical → clean
→ data quality gate → rules engine → calculate → validate against golden
→ reconcile → classify variance → detect anomalies → exception register
```

Full detail, including how each stage would map onto Azure Data Factory, Synapse and Airflow in an enterprise build, is in `docs/architecture.md`.

Documentation

Document	Contents
<code>docs/business_requirements.md</code>	Client objective, 20 functional requirements, risks, acceptance criteria
<code>docs/architecture.md</code>	Pipeline, layer contracts, cloud mapping, ETL vs ELT
<code>docs/skill_matrix.md</code>	Role requirements mapped to project evidence
<code>docs/study_plan.md</code>	Seven day plan
<code>docs/day01.md</code>	Day 1 concepts and exercise

Tech

Python, pandas, Polars, DuckDB, SQL, pytest, YAML configuration, Git, GitHub Actions. Runs entirely locally. No paid services, no cloud account required.

Key engineering decisions

1. **Ground truth first, corruption second.** The generator computes the

correct tax, stores it, then damages the source data. Every exception the pipeline raises is therefore checkable against a known answer.

1. **Rules as configuration.** A tax analyst changes a YAML decision table. No

Python change, no deployment, and the regression suite proves nothing else moved.

1. **Effective dating throughout.** Rates are resolved as at the transaction

date, never today's date. Repricing history with a current rate is one of the most expensive bugs in tax data.

1. **Half-up rounding, explicitly.** Python's `round()` is banker's rounding.

Finance is not. The generator, the engine and the tests all use `Decimal(ROUND_HALF_UP)`.

1. **Raw data is immutable.** Everything is re-derivable from the landing zone,

which is what makes a six-month-late rate correction possible.